L04 – Bash Scripting - Part 2

1.0 Control Structures in Bash

Last time we worked on the basics of putting together a Bash script. Now, it is time to add to this the control structures that actually make scripting useful. Control structures are means of changing the flow of a shell script (or a program) based on certain conditions. So, control structures do things like:

Control structures are means of changing the flow of a shell script (or a program) based on certain conditions. So, control structures do things like:

```
if [condition A] then
    [execute a series of instructions]
else if [condition B]
    [execute a different series of instructions]
else [if conditions A or B don't exist]
    [do something else]
end
```

Going from the first statement to the end defines what we call a "block".

There are other control structures too, for instance:

```
while [condition exists]
    [do something]
end
```

or

```
for [condition]
    [do something]
end
```

and so on. These control structures enable one to loop over all files in a directory, make decisions based on certain conditions, and only operate instructions should a particular condition exist. The code I have written above is what is called pseudocode, which is basically a bunch of English statements that mimic the structure of the code to be written, but are designed to be read by humans. Writing pseudocode can be a useful way to design a new shell script, because it forces you to think about the design of the control structures and not focus on the details and correct formatting of each command.

Control structures are based around these commands:

- if
- while
- for

- until
- case

and some commands that work well with these are:

- break
- continue
- exit

2.0 If-then-else Control Structure

Consider if I want to write a script for someone to see if they can guess my lucky number (6). A simple version would be like this.

```
#!/bin/bash -xv

echo "enter a number from 1 to 10"
read guess
if [ $guess -eq 6 ]
then
    echo "you guessed it"
else
    echo "$guess is incorrect"
fi
```

The first two lines in the script should look familiar to you, so let's consider the third:

if [$guess -eq 6]

This statement says that if the variable guess is equal to 6, then execute the commands between **then** and **else**. If the guess is not equal to 6, then execute the commands between **else** and **fi**. **fi** is bash's way of ending a control block that starts with **if**.

Bash is picky about the formatting of an if statement. There must be a space between the opening bracket **[** and the first part of this test, which in this case is **$guess**. To test if **$guess** is equal to a number, we use the word **-eq**. However, if you wanted to test if **$guess** is the same as a word, you'd use the word **=**. So, we could write a script like this if we wanted to match a string.

```
#!/bin/bash -xv

echo "guess my favorite animal"
echo "enter an animal"
read guess
if [ $guess = "lion" ]
then
    echo "you guessed it"
else
    echo "$guess is incorrect"
```

```
fi
```

In other words, it's important to distinguish if you are testing for numerical equality or string equality.

Let's make our initial example a little more complicated. We'll give a hint and do some error checking.

```
#!/bin/bash -xv

echo "Enter a number between 1 and 10… "
read guess
echo "guess is $guess"
echo ""

if [ $guess -gt 10 -o $guess -lt 1 ]
then
  echo "you didn't pick a number between 1 and 10!"
  echo "follow the instructions and try again..."
elif [ $guess -lt 6 ]
then
   echo " your pick is too low"
elif [ $guess -gt 6 ]
then
  echo "your pick is too high"
else
  echo "you got it!"
fi
```

Let's look through this. The first if determines if **$guess** is greater than (**-gt**) 10 or (**-o**) less than (**-lt**) 1. **elif**, is a way of saying else if, and means that if the first if statement is not true, then we'll check if **$guess** is less than 6. If not, we drop down to the next **elif** statement, which checks if **$guess** is greater than 6. If none of these are true, then **$guess** must be 6.

Here's a table of some tests that can be done with if and other control structures, from http://www.codecoffee.com/tipsforlinux/articles2/043.html

Checks equality between numbers	
x -eq y	Check is x is equal to y
x -ne y	Check if x is not equal to y
x -gt y	Check if x is greater than y
x -lt y	Check if x is less than y

Checks equality between strings	
x = y	Check if x is the same as y
x != y	Check if x is not the same as y
-n x	Evaluates to true if x is not null
-z x	Evaluates to true if x is null

One can also do other tests, like test for the existence of a file. For instance, one could do

if [-e $file]

to test for the existence of a file. I use this sort of thing a lot to see if a directory exists, e.g. :

```
if [ -d $directory ]
then
   mkdir $directory
fi
```

Here's a big old table of things one can do with if and files, from the Bash Beginner's Guide (http://tldp.org/LDP/Bash-Beginners-Guide):

Primary	**Meaning**
[-a FILE]	True if FILE exists.
[-b FILE]	True if FILE exists and is a block-special file.
[-c FILE]	True if FILE exists and is a character-special file.
[-d FILE]	True if FILE exists and is a directory.
[-e FILE]	True if FILE exists.
[-f FILE]	True if FILE exists and is a regular file.
[-g FILE]	True if FILE exists and its SGID bit is set.
[-h FILE]	True if FILE exists and is a symbolic link.
[-k FILE]	True if FILE exists and its sticky bit is set.
[-p FILE]	True if FILE exists and is a named pipe (FIFO).
[-r FILE]	True if FILE exists and is readable.
[-s FILE]	True if FILE exists and has a size greater than zero.
[-t FD]	True if file descriptor FD is open and refers to a terminal.
[-u FILE]	True if FILE exists and its SUID (set user ID) bit is set.
[-w FILE]	True if FILE exists and is writable.
[-x FILE]	True if FILE exists and is executable.
[-O FILE]	True if FILE exists and is owned by the effective user ID.
[-G FILE]	True if FILE exists and is owned by the effective group ID.
[-L FILE]	True if FILE exists and is a symbolic link.

Primary	Meaning
[-N FILE]	True if FILE exists and has been modified since it was last read.
[-S FILE]	True if FILE exists and is a socket.
[FILE1 -nt FILE2]	True if FILE1 has been changed more recently than FILE2, or if FILE1 exists and FILE2 does not.
[FILE1 -ot FILE2]	True if FILE1 is older than FILE2, or is FILE2 exists and FILE1 does not.
[FILE1 -ef FILE2]	True if FILE1 and FILE2 refer to the same device and inode numbers.
[-o OPTIONNAME]	True if shell option "OPTIONNAME" is enabled.
[-z STRING]	True of the length if "STRING" is zero.
[-n STRING] or [STRING]	True if the length of "STRING" is non-zero.
[STRING1 == STRING2]	True if the strings are equal. "=" may be used instead of "==" for strict POSIX compliance.
[STRING1 != STRING2]	True if the strings are not equal.
[STRING1 < STRING2]	True if "STRING1" sorts before "STRING2" lexicographically in the current locale.
[STRING1 > STRING2]	True if "STRING1" sorts after "STRING2" lexicographically in the current locale.
[ARG1 OP ARG2]	"OP" is one of -eq, -ne, -lt, -le, -gt or -ge. These arithmetic binary operators return true if "ARG1" is equal to, not equal to, less than, less than or equal to, greater than, or greater than or equal to "ARG2", respectively. "ARG1" and "ARG2" are integers.

One can combine these operators using the operators listed below (again from Bash Beginner's guide). These work in decreasing order of precedence from top to bottom.

Operation	Effect
[! EXPR]	True if **EXPR** is false.
[(EXPR)]	Returns the value of **EXPR**. This may be used to override the normal precedence of operators.
[EXPR1 -a EXPR2]	True if both **EXPR1** and **EXPR2** are true.
[EXPR1 -o EXPR2]	True if either **EXPR1** or **EXPR2** is true.

For what it's worth, I don't remember all of these. I actually write shell scripts pretty infrequently because of other job commitments. I find the key to shell scripting successfully is just to have a rough idea that something can be done in Bash; then I write some pseudocode and work out the details by googling various bash guides on-line.

2.1 exit

The **[** *some logic statement* **]** construct is called a *test command*. It evaluates to 0 or 1, depending on whether the statement is true (evaluates to 0) or false (evaluates to 1). The 0 or 1 value is returned in the exit status of the expression, and to get at it, you can type

echo $?

So one way to see if a test command is doing what you expect is to type it in the command line, for instance:

```
>>b=5
>>[ $b -eq 5]
>>echo $?
```

If the result is 0, the test command was true. The value returned is called the exit status. Not only does a test command return an exit status, but every command does, 0 if it is successful, and some non-zero value that represents an error code if it does not.

If you are writing a shell script, and you want to give an indication that something has gone wrong you can have it terminate with **exit 1**.

You can also perform a test command with **test**. We won't worry about that here.

Note that in Matlab, if a logic statement is true, it will return a value of 1, just the opposite of Bash.

2.2 Extended test commands.

Bash also has a more flexible set of test commands that follow a format closer to other programs. For numbers, one can use:

((***some test*** **))**

and for other stuff, use

[[***another test*** **]]**

I'm just telling you so you know more about this in case you find yourself wishing for something more sophisticated. Here's a source that explains more:
http://tldp.org/LDP/abs/html/testconstructs.html.

3.0 Loops

Loops are a control block of commands that repeatedly are executed until a condition is met. For instance, one could loop through all of the files in a directory and process them (I do this all the time to process seismic data). Looping commands are while, until, and for.

3.1 While-do-done

The while command tells a block of code to run over and over, as long as the test construct associated with the while command is true.

Let's say you want to modify your number guessing script so that a person can keep guessing until they guess correctly. Let's write some pseudocode

```
Input guess
guess_flag = "incorrect"
While [guess_flag is incorrect]
        # See if guess is correct
        if guess > answer
                echo "guess too high"
        else if guess < answer
                echo "guess too low"
        else  # guess must be correct
                echo "you did it!"
                guess_flag = "correct"
        fi
done
```

When writing pseudocode, just roughly approximate control statements. The goal is to get the flow of the script correct. Now, let's write this in a proper Bash script. While means to keep executing the following block as long as the pattern in the while statement is true. For if we used **if**-**then**-**fi** to produce the **if** control block. For **while**, it's **while**-**do**-**done**, as you can see below:

```
#!/bin/bash

echo "Enter a number between 1 and 10… "
read guess
guess_flag="incorrect"
while [ "$guess_flag" = "incorrect" ]
do
  if [ "$guess" -gt "6" ]
  then
     echo "  your pick is too high, guess again"
     echo -n "Input a new guess: "
     read guess
  elif [ "$guess" -lt "6" ]
     then
     echo "  your pick is too low, guess again "
     echo -n "Input a new guess: "
     read guess
  else
     echo "  you got it!"
     guess_flag="correct"
  fi
done
```

Note I used echo –n for the input. Type

>> **man echo**

to see what this does.

3.2 Until-do-done

until is essentially the opposite of **while**. It tells the following block to continually repeat until the test condition is true. We could rewrite our shell-script above like this.

```
#!/bin/bash
echo "Enter a number between 1 and 10… "
read guess
until [ "$guess" –eq "6" ]; do
  if [ "$guess" -gt "6" ]; then
     echo "  your pick is too high, guess again"
     echo -n "Input a new guess: "
     read guess
  elif [ "$guess" -lt "6" ]; then
     echo "  your pick is too low, guess again "
     echo -n "Input a new guess: "
     read guess
  fi
done
echo "  you got it!"
```

This script will run until you guess the correct number. You may note that I did something a little different from the last script: I put two commands on the same line, separated by a semicolon. The semicolon separates two commands. You can do this on the command line too, for instance, you can type

```
>> ls; who
```

And when you hit return both commands will execute, as if you typed

```
>> ls
>> who
```

So, rather than type:

until ["$guess" –eq "6"]
do

I put these on the same line:

until ["$guess" –eq "6"]; do

You can do things either way. I personally find the latter way to be neater, but it's a matter of personal preference.

3.3. For-do-done

for is a little more complicated than **while** and **until**. It loops over a list, and after each iteration, it takes a variable and changes the variable value to the next member of the list.

It can loop over a range of numbers, for instance, here it goes from 1 to 10, and c goes up by 2 each time. In this case the list is [1,3,5,7,9].

```
#!/bin/bash -xv
for (( c=1; c<=10; c=c+2)) ; do
   echo "c = $c"
done
```

You could also do it this way

```
#!/bin/bash -xv
for c in {1..10..2}; do
   echo "c=$c"
done
```

This won't work for bash versions earlier than 4.0. You can figure the bash version out by typing:

```
>> echo $BASH_VERSION
```

Where I find for to be really handy, is if I populate the list will all the files in a directory. This script will list all the files in the directory in which you invoke the script.

```
#!/bin/bash -xv
for file in * ; do
   echo "file is = $file"
done
```

This becomes really powerful if you have a series of commands you want to run on each file.

Now, say you want to do something to some files, but not others. The **continue** command skips to the next iteration of the loop. In the following example, if the file meets a condition, the loop will skip to the next iteration. If it doesn't it will execute a commands. To get a feel for this, make 3 empty files.

```
>> echo " " >> file1.txt
>> echo " " >> file2.seismogram
>> echo " " >> file3.txt
```

```
for file in * ; do
   echo " "
   echo "file is = $file"
   file_suffix=`echo $file | cut -d. -f2 `
   echo "file_suffix = $file_suffix"
```

```
   if [ "$file_suffix" == "seismogram" ] ; then
       echo "$file is a seismogram: let's process this file!"
       continue
   fi
   echo "$file is not a seismogram, will ignore"
done
```

Another useful command is break. This works with while and until too. If break is invoked, the control sequence stops. Here's an outline of how it might work.

while : # make infinite loop
do
if [*condition is met*]
break
fi
done

This will loop until a condition is met, and then stop.

4.0 Switch Case

This is a really nice structure that is similar to an **if then** type of structure. Suppose I wanted to do some action based on what kind of seismic arrival I was looking at. I could use the switch case structure

```
#!/bin/bash -xv
input_phase=$1
case "$input_phase" in
  PKP)
    echo "PKP arrival"
    ;;

  SKS)
    echo "SKS arrival"
    ;;

  SPdKS)
    echo "SPdKS arrival"
    ;;

esac
```

5.0 The Dialog utility

Let's wrap up our lectures on Bashl scripting with an entertaining utility. Perhaps you want to impress your advisor and make him/her think you've already developed these mad hacking skills. Well, try asking for input using the dialog utility.

You may have to install it. If typing display on the command line results in a "command not found", then just

```
>> su
[type root password: th!s!sth33nd]
>>yum install display
```

Here's quick demo of display:

```
#!/bin/bash

dialog --title "-----  WARNING  -----" \
--infobox "This computer will explode \
unless you press a key within the next 5 seconds!" 7 50;
exit_status=$?
```

The dialog utility uses the following syntax:

```
dialog --title {title} --backtitle {backtitle} {Box options}

where Box options can be one of the following (other options also exist
if you check out the man page)
 --yesno      {text}  {height} {width}
 --msgbox     {text}  {height} {width}
 --infobox    {text}  {height} {width}
 --inputbox   {text}  {height} {width} [{init}]
 --textbox    {file}  {height} {width}
 --menu       {text}  {height} {width} {menu} {height} {tag1} item1}...
```

Here is an example of how to create a yes/no box:

```
#!/bin/bash
ifile='blah.txt'

dialog --title "-----  Yes/No Example  -----" \
  --yesno "Do you want to delete file $ifile" 7 60

exit_status=$?   # get the dialog utilities exit status

echo " "

case "$exit_status" in

  0)
  #user selected 'yes'
  echo "Deleting file $ifile"
  #rm $ifile
  ;;


  1)
  #user selected 'no'
```

```
  echo "Saving file $ifile"
  ;;

  255)
  #user hit escape key
  echo "Operation Canceled…"
  ;;

esac
```

6.0 Parting Comments

As your bash scripting, or other coding, needs become more sophisticated, it becomes increasingly useful to take a more structured approach. Some advice on how to deal with complexity and work flow follows.

6.1 Goals

First, consider the goals of your project. What steps do you need to get there? Generally, research takes unexpected directions, since the nature of research is that you are exploring something new to human knowledge. However, given what you know, you should still should have a broad outline of the tasks you need to complete to bring your project to fruition.

In seismology we do a lot of data processing and file manipulation. Often, we find something that needs to be fixed, or an assumption that we made was incorrect, and aspects of data processing need to be repeated. The best way to do this is keep your unprocessed data in a directory, and for each processing step, move the increasingly processed data into a new directory. For each step in your processing, consider writing a new shell script. Ideally, you'll be able to write a master shell script that runs a lot of individual shell scripts so that you can always duplicate all of the processing you've done by the press of a button. Sometimes, for instance when you are manually identifying useful seismograms, this can't easily be done, but it's a good ideal to keep in mind.

Make sure it is clear what the nature is of any seismograms or other data files in each directory. Consider adding a README text file in each directory, or in some prominent location, so you, a future student, or your advisor can revisit what has been done.

6.2 Planning your code

Things to think about:

- What are the inputs and outputs of your code? Often, I modify the file name of a seismogram to reflect where it is in the processing stream.

- What other code or shell scripts need to be run before this script or code runs? What needs to be done afterwards? What assumptions are you making about the files you are manipulating?
- It's often best to outline what you want to do in pseudo-code. This is a series of simple English-language statements in which you describe the looping and tasks of each part of your code. Once you've done this, then you can start writing your code and worrying about getting syntax correct.
- Flexibilty versus time commitment--think about if the code will be used more than once and whether it will be useful to make the code flexible enough to be used on different data or for related tasks. At the same time, you are not a professional programmer, so keep in mind your ultimate goal is to arrive at a research result, not to produce good code.

6.3 Commenting your code

For code that is much more than 20-30 lines in length, add comments that describe: 1) the goals of the code; 2) where the code is in a larger processing stream, such as what code or shell scripts need to be run before this code; 3) what assumptions you are making (say that all the seismograms you are processing have the same length); and 4) where the input and output files will reside, if you have any.

Keep in mind that others will want to use your code too. Most of the research we do here at CSU has been supported in some way with public resources, and hence I think we should be open with sharing our code with our colleagues.

Most graduate students tend to be too disorganized. Remember, the goal of a scientist is that all research should be as reproducible as possible. That said, there are some students that get so involved in the process of coding that they forget the overall goal of their projects. Keep in mind that the end goal at all times, and don't spend days commenting or optimizing code.

6.4 There's more!

This is the end of our bash scripting lab. We've just covered the most basic stuff. Perhaps the only other fundamental concept in bash scripting, and unix in general, that I haven't covered is regular expressions, or regex. This is a method that one can use to match patterns in a string. It's very powerful, and also rather arcane. We don't really have time to go into this, but you can google "regular expression tutorial" to learn more.

Likewise, if you run into problems with loops, awk, or other commands, just google "awk example", and a wealth of information will appear. I forget a lot of the bash commands, and typically re-learn them via google, as I need to.

7.0 Homework

1) Write a script that will allow a user to guess if she or he has picked your favorite number, which will be between 0 and 200:

- The user should get a hint of they need to guess higher or lower than their previous guess.
- The user should also get a hint if they are close—within 20 of the number.
- This should loop until they guess correctly or quit.
- After every guess, tell the user how many total guesses they have made.
- After every fifth guess, give the user an option to quit. You can get the remainder of a division using this command: let a=$num_guesses%5. In this case, if $a –eq 0, then $num_guesses is a multiple of 5.

Send the bash script to the usual location, and call it Lastname_Firstname_Lab04.bash.